Hiking Assistant - user manual

This manual explains how to set up and use the Hiking Assistant system. Target hardware was not available for end-to-end verification of the motion stream; check the watch and Pi together before relying on field data.

1. What the system shows

The watch shows a live hardware step count. After STOP, it shows an estimated distance and offers UPLOAD or NO. While a session is active, the watch sends three-axis acceleration to the Pi at a nominal 25 samples per second. The Pi writes a CSV and, if a compatible model is installed, classifies 2-second windows into `idle`, `walking`, `climbing`, or `running`. The browser shows saved session steps, distance, estimated calories, dominant activity, and the share of classified windows. The browser does **not** show live activity.

`climbing` uses upstairs training data as a proxy; `running` uses jogging data. The model was trained on a different wrist sensor and has not been validated on this watch. Treat labels and calories as indicative only.

2. Prepare the Raspberry Pi

Use a Raspberry Pi 3B+ with Raspberry Pi OS, working Bluetooth, Python 3.9 or newer, and the system dependencies needed to build `bluepy` (including GLib development headers). From the repository root:

```
cd RPi
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -r requirements.txt
```

The model is a separately generated local file, `RPi/activity_model.pkl`. To reproduce it from SELFBack public wrist data, download the archive and run the training command in the main README. Check dataset terms before redistribution. Use the same scikit-learn version (`1.6.1`) for training and on the Pi. A missing model does not stop step upload or CSV recording; saved activity becomes `unknown`. Never load an untrusted pickle.

Find the watch Bluetooth MAC address with a BLE scanner or `bluetoothctl scan on`; set `HIKING_WATCH_MAC` to that address. The configured fallback MAC may not match your watch. The Pi receiver connects by address and subscribes to notifications; it does not require a separate manual pairing step.

3. Build the watch firmware

Use an ESP32 Arduino toolchain configured for TTGO T-Watch 2020 V3 and LilyGO TWatch Library 1.4.2. Open `Watch/hikingassistant_RTOS/hikingassistant_RTOS.ino`; the implementation resides in the adjacent `hikingassistant.cpp`. Select the board and serial port, then build and flash. Verify flashing and runtime behavior on the actual device.

4. Start the services

Open two terminals in `RPi`, each with the virtual environment activated:

```
export HIKING_WATCH_MAC='AA:BB:CC:DD:EE:FF'
python receiver.py

python wserver.py
```

Replace the MAC with the watch's address. The receiver prints connection status and writes detailed messages to `RPi/receiver.log`. `RPi/setup.sh` can launch both services in the background, with stdout/stderr in `RPi/logs/`, but use the foreground commands first when diagnosing a setup. The script does not install packages or train the model.

The web server listens at `http://127.0.0.1:5001` on the Pi by default. If you deliberately need access from another device on a **trusted** private network, set `HIKING_BIND` to the Pi's network address and browse to that address on port 5001. Do not expose this prototype to the public internet. The default account initialized by the receiver is username `user123`, password `user123`. This is a development account with plaintext storage; change the authentication design before real deployment.

5. Record and upload a session

1. Charge and power on the watch; confirm the Hiking Assistant display appears. Keep it near the Pi and wait for the receiver's "Connected to Watch" message.
2. Tap green **START**. The watch resets the hardware step counter and begins nominal 25 Hz motion sampling. The step count appears on the watch.
3. Tap red **STOP** to finish. The watch displays steps and estimated distance. Sampling stops; motion samples already received stay in the Pi CSV.
4. Tap **UPLOAD** to send the final step summary. Wait for "Uploaded" on the watch. The receiver stores the session in SQLite and acknowledges the watch. If BLE is disconnected, the watch shows "No BLE connection" and returns to the upload choice; reconnect and retry.
5. Tap **NO** to discard the session summary instead. This does not erase motion samples already streamed to the Pi.
6. Open `/home` after login. Sessions appear newest first. Activity distribution describes prediction windows, not precise time spent in each state. `UNKNOWN` means no valid predictions were available when the session was saved.

Only one unfinished step session is held on the watch. The watch does not retain a raw-motion backlog and cannot resend lost stream packets. BLE gaps and receiver queue overflow may leave holes in CSV or reset a partial inference window.

6. Offline test without hardware

From the repository root with requirements installed:

```
python -m unittest discover -s tests -v
```

With a locally trained model and SELFBack archive, export and replay a sample:

```
python RPi/export_public_sample.py /path/to/selfback.zip /tmp/walking.csv --subject 008 --label walking
python RPi/replay.py /tmp/walking.csv --model RPi/activity_model.pkl
```

Replay checks the data path, not watch accuracy. Public data came from an Axivity AX3 sensor. The current held-out results and known walking/upstairs confusion are in `MODEL_EVALUATION.md`.

7. Files and troubleshooting

- `RPi/recordings/*.csv`: one received motion stream file per connection with samples; columns are sequence, watch uptime in ms, X/Y/Z in g, watch drop count, Pi receive time in UTC, and a gap marker. Use Pi receive time when labeling sessions; the watch clock is uptime only. These files can grow during long sessions. Back up or remove old files deliberately.
- `RPi/sessions.db`: SQLite session summary and prototype login database. Existing databases are migrated automatically when the receiver starts.
- `RPi/receiver.log`: connection, prediction, and gap messages. `RPi/logs/` contains output if `setup.sh` was used.
- `RPi/activity_model.pkl`: trusted local classifier. The model and raw recordings are ignored by Git.

If the watch will not connect, confirm MAC address, Bluetooth power, range, and that no other central holds the connection. If the web page is unavailable, confirm `wserver.py` is running and inspect its terminal/log; remote access also requires an intentional `HIKING_BIND` setting. If activity is `UNKNOWN`, verify the model file path and its feature metadata/version, then inspect `receiver.log`. If motion CSV has gaps, inspect `sequence`, `watch_time_ms`, `watch_dropped`, and receiver logs; connectivity or bounded queues can lose data. A session without valid activity predictions legitimately shows `UNKNOWN`.

8. Field validation checklist

Before relying on a real watch: measure static acceleration magnitude and axis orientation, compare sample timestamp intervals with 40 ms, verify step reset and count across short controlled walks, test STOP/UPLOAD/NO and BLE reconnect, monitor drop counters and CSV continuity, and run a long session to assess storage and battery. Label representative watch recordings from multiple people before claiming activity accuracy or retraining for this hardware.